

Q&A; Elite Reference — defend every line

Zomato Delivery · North vs South Regional Classification · Team Hmmmmmmmmmm · 45+ questions, deep answers

Reading order. Section A (dataset + labels) explains *what* we classified. Section B explains *every statistical test* we used with formula + our specific result. Section C unpacks *each trained model* — features, hyperparameters, AUC. Section D explains *cross-validation* in three forms. Section E walks through *every stress test*. Section F is the *interpretation layer*. Sections G–I cover *adaptability, defense, limitations*. If you read only one section before pitching, read **F**.

Section A. Dataset and labels

Q. What is the dataset?

A. 45,584 rows × 20 columns of Zomato food-delivery orders collected Feb 11 – Apr 6, 2022 across 22 Indian cities. Columns include rider ID, age, rating, order/pickup timestamps, restaurant and customer coordinates, weather, traffic density, vehicle type, multiple-delivery count, festival flag, city tier, and the target Time_taken (min). After cleaning we work with 41,944 rows in North/South (16,397 South · 25,547 North).

Q. How is the 'South India' label defined?

A. Per the team brief: **South = Kerala, Tamil Nadu, Andhra Pradesh, Telangana, Karnataka. North = every other state** including Maharashtra, Gujarat, Goa, MP, Rajasthan, UP, Punjab, Uttarakhand, Jharkhand, West Bengal. Goa folded into North per spec.

Q. Where does the ground-truth label come from?

A. Each Delivery_person_ID has a city prefix: KOCRES16DEL01 → KOC = Kochi → Kerala → South. A 1-line regex pulls the prefix; a 22-row dictionary in src/regional.py maps prefix → city → state → region. No reverse-geocoding required.

Q. How did you verify the city-code map is correct?

A. Coordinate cross-check: for each known city code, compute the median Restaurant_lat / Restaurant_lon of its orders and compare to a known centroid for that city. All 22 cities passed the 250-km tolerance; worst offender is Goa at 45 km from centroid; 19 of 22 are within 7 km. See reports/regional_city_check.csv.

Q. What if a city code is unknown in production?

A. _classify_by_coords() in src/regional.py uses a bounding box (South: 8°-19°N, 74°-84°E; North: 18°-32°N, 68°-89°E) as a fallback. Rows still unmapped after both passes are flagged 'Unknown' and dropped from training so they're never silently mislabelled.

Q. Why include Andhra Pradesh as 'South' if no AP city appears in the data?

A. The brief defines AP as South. Our mapping handles it correctly — if an AP city code arrives later, it will route to South via the state dictionary. We're future-proof, not over-fit to the current 22.

Section B. Statistical tests we used

For each test: when to use it, the test statistic, what p-value means, our specific result.

Q. What is the Mann-Whitney U test?

A. A **non-parametric** test for whether two independent samples come from the same continuous distribution. Equivalent to asking if the medians are equal when the shapes are similar. Statistic U counts the number of ranked-pair comparisons where sample-1 > sample-2. **Why we use it:** the data is not Gaussian (Time_taken, distance, age all have heavy tails), so a t-test would be unsafe. Mann-Whitney makes no distributional assumption. **Our result on F1 (delivery time):** $p = 2.13e-01$ — **not significant**. Median delivery time is essentially identical North vs South.

Q. What is the Chi-square (χ^2) test of independence?

A. Tests whether two categorical variables are independent. Build a contingency table (rows = categories of variable 1, cols = categories of variable 2). Under independence, expected cell count is (row total $\times$ col total) / grand total. χ^2 is the sum of (observed - expected)² / expected. Large χ^2 + small p-value = the two variables are dependent. **Why we use it:** weather, traffic, vehicle type, etc. are categorical. We want to know if their distribution differs by region. **Our result on F3 (traffic):** $\chi^2=1.0$, $p = 7.95e-01$ — **not different**. Traffic density distribution is the same North vs South.

Q. What is the Kolmogorov-Smirnov (KS) two-sample test?

A. Compares two **entire distributions** (not just medians). Statistic D is the maximum vertical distance between the two empirical cumulative distribution functions (CDFs). **Why we use it:** Mann-Whitney can miss shape differences (e.g. same median but one distribution has fat tails). KS catches that. **Our key result:** distance_km has KS D = 0.088, $p \approx 6.7 \times 10^{-3}$ — distributions differ in *shape* even though Mann-Whitney medians match (south 9.2 vs north 9.0 km). This is the only column that fails KS — and it's the one that ends up carrying the regional signal.

Q. What is ANOVA (analysis of variance), and why use it?

A. Tests whether the means of three or more groups are equal. Statistic F = (between-group variance) / (within-group variance). Used in the EDA on traffic density (4 groups: Low/Medium/High/Jam) vs Time_taken (regression task). **Result there:** $F=3,390$, $p \ll 10^{-3}$. Traffic groups have very different median times. We did NOT run ANOVA on the regional task because we have only 2 groups (N vs S) — Mann-Whitney is the correct choice.

Q. What is Welch's t-test and where did we use it?

A. Variant of the two-sample t-test that does NOT assume equal variance between the two groups. We used it on F4 in the morning ETA EDA (festival vs non-festival times). Conservative when sample sizes / variances are unequal. In the regional task we preferred Mann-Whitney for non-parametric safety.

Q. What is Spearman correlation (ρ)?

A. Pearson correlation computed on the **ranks** of the variables instead of the raw values. Captures monotonic (not necessarily linear) relationships. Robust to outliers. $\rho = 1$ = perfect rank order, 0 = no monotonic relationship, -1 = perfectly inverse. **Example:** distance $\leftrightarrow$ delivery-time $\rho = 0.32$ in the original ETA dataset — moderate monotonic relationship.

Q. What is Cramér's V, and why is it needed alongside Chi-square?

A. Effect-size metric for Chi-square. $V = \sqrt{\chi^2 / (n \cdot (\min(\text{rows}, \text{cols}) - 1))}$. Range 0 to 1. **Why needed:** with 45,000 rows even a tiny dependency yields a very small p — V tells you whether the effect is large enough to care about. Our F2 weather: V was small (around 0.02), confirming the distributions may be statistically distinct in some cuts but operationally indistinguishable.

Q. What does $p < 0.05$ actually mean?

A. Under the null hypothesis (no real effect), the probability of observing a test statistic at least as extreme as ours is 5% or less. It is **not** the probability that the null is false. With 45,000 rows even a tiny real effect can yield $p < 0.05$ — which is why we report effect sizes (Cramér's V, KS D, ρ) alongside p-values.

Q. What is the haversine formula?

A. Great-circle distance between two points on a sphere given latitude/longitude. With $R = 6,371$ km:

```

$$\Delta\phi = \text{lat}_2 - \text{lat}_1, \Delta\lambda = \text{lon}_2 - \text{lon}_1 \text{ (radians)}$$

$$a = \sin^2(\Delta\phi/2) + \cos(\text{lat}_1) \cdot \cos(\text{lat}_2) \cdot \sin^2(\Delta\lambda/2)$$

$$d = 2 \cdot R \cdot \arcsin(\sqrt{a})$$

```

Section C. Trained models — what each one does

Name	File	Features	Mean AUC	Purpose
Model A — GEO	(none saved; recreatable)	5 geo: Restaurant lat/lon, Delivery lat/lon, distance	0.600	Calibration — proves labels are clear
Model B — Behavioural	(none saved; not saved separately)	12 num + 6 cat behaviour features, 38 cat	0.588	Revealed the employable week-month domain
Model B' — Behavioural (structured)	models/region_classifier_v1.txt (~350 KB)	Model B but month + day_of_week removed	0.508	Best behaviour-only baseline; essential
Model C — Structural	models/region_classifier_v2_structural.txt (~1.9 MB)	Model B' (+ distance)_km (haversine)	0.596	The structural finding — geometric

Q. How is each model trained?

A. Identical recipe across all four:

```
objective = binary (logistic loss, sigmoid output)
metric = AUC
n_estimators = 400-800 with early_stopping_rounds = 30-40
learning_rate = 0.04 - 0.05
num_leaves = 31 - 63 (leaf-wise growth)
min_child_samples = 20 (prevents tiny leaves)
class_weight = 'balanced' (corrects 39/61 imbalance)
random_state = 42
categorical_feature = passed natively, no one-hot
```

Q. What does 'class_weight=balanced' do?

A. Re-scales each sample's loss inversely to its class frequency. North gets weight ≈ 0.82 , South gets ≈ 1.28 , so the model can't get a low loss by always predicting 'North'. Equivalent to oversampling the minority class.

Q. What does 'early stopping' do?

A. After every tree we add, we measure AUC on the held-out validation fold. If AUC stops improving for 40 consecutive trees, we stop adding trees and keep the best iteration. Prevents overfit and keeps model size adaptive.

Q. Why LightGBM specifically?

A. Five reasons. (1) **Native categorical handling** — finds optimal subset splits over weather, traffic, vehicle, city, festival without one-hot bloat. (2) **Native NaN handling** — missing rider ratings stay as NaN, the model learns the missing-direction split. (3) **Leaf-wise growth** — focuses splits on high-loss leaves, trains fast on tabular data. (4) **Histogram-based** — feature binning makes training $O(\text{features} \cdot \text{bins})$ instead of $O(\text{features} \cdot \text{rows})$. (5) **Reproducible** — same seed $\rightarrow$ same model.

Q. Why also test with CatBoost?

A. Independent gradient-boosting library (Yandex). Different default categorical encoding (ordered target statistics instead of LightGBM's gradient-based). If our finding survives *both*, it isn't a framework-specific artefact. **Result:** CatBoost on Model B' gave AUC = 0.507 — within 0.001 of LightGBM's 0.508.

Q. How do you LOAD the saved model in Python?

A. Three lines, no training required:

```
import lightgbm as lgb
booster = lgb.Booster(model_file='models/region_classifier_v2_structural.txt')
p = booster.predict(X) # probability of being SOUTH for each row
```

Live demo for the judges: open a terminal in the repo root, run the snippet above. `booster.num_trees()` prints ~ 600 . `booster.feature_name()` prints the 17 features used. That's tangible

proof the model is real, not a slide claim.

Section D. Cross-validation — three forms, three answers

Q. What is KFold cross-validation?

A. Split the dataset into K equal pieces (folds). Train on K-1 folds, predict on the held-out fold. Rotate so every row gets one out-of-fold prediction. Average the K validation scores for a single CV metric. We use K = 5 with `random_state = 42` everywhere so deltas across models / runs are honest.

Q. What is StratifiedKFold and why prefer it here?

A. KFold that preserves the class ratio within each fold. With our 39/61 South/North split, plain KFold could randomly produce a fold with 25/75 by chance, distorting the AUC. Stratified guarantees ~39/61 in every fold.

Q. What is GroupKFold and when do we use it?

A. KFold variant where rows from the same **group** (here: same `city_code`) are never split across train and test. **Why critical:** with plain StratifiedKFold the model sees most cities in both train and test, which lets it memorise per-city patterns. GroupKFold trains on say 17 cities and tests on 5 unseen cities — answering 'does the pattern generalise to a city we never trained on?' **Our key result:** behaviour-only GroupKFold AUC = 0.506 confirms no behavioural signal; structural GroupKFold AUC = 0.911 confirms the `distance_km` signal IS generalisable.

Q. What are out-of-fold (OOF) predictions?

A. For every row, the prediction made when that row was in the validation fold. Concatenating all five fold predictions gives one honest score per row, never trained on itself. We use OOF for the final confusion matrix, F1, and ROC curves.

Section E. Stress tests — what each tried to break

#	Test	What it checks	Result
1	KS shape on every numeric column	Distributions differ in shape, not just median (could be identical)	Only distance_per_age is significant (p=0.0088, p=10 ⁻¹⁰)
2	Dispersion ratio south_std / north_std	Identical medians could hide different spreads (variance)	All features within 0.98 - 1.04 — no spread difference
3	Engineered interaction features	Pairwise crosses (traffic×hour, weather×traffic, vehicle×multi)	4 of 5 interactions add zero AUC, none significant
4	GroupKFold by city	Behaviour signal must generalise to UNSEEN	Behaviour just stays at 0.506, not a local bias
5	Per-South-state 4-way classifier	Is South itself internally uniform across Karnataka, Kerala, Tamil Nadu, Andhra Pradesh?	Auc: 0.97 Kerala, 0.259? — weakly heterogeneous
6	CatBoost cross-check	Is the 0.508 result a LightGBM-specific artefact?	CatBoost AUC = 0.507 — confirms within 0.001

Q. Why six stress tests instead of just reporting the main model?

A. Because the headline ('regions are operationally identical') is a NEGATIVE finding — easier to dismiss as 'you didn't look hard enough'. Each stress test attacks a specific way the negative could be wrong: (1) hidden shape differences, (2) hidden variance differences, (3) hidden interactions, (4) per-city overfit, (5) within-South heterogeneity, (6) framework artefact. Surviving all six is what makes the conclusion defensible.

Q. What did the ablation actually show?

A. Added one engineered feature at a time on top of base behaviour, ran GroupKFold each time:

Added feature	GroupKFold AUC	Lift
distance_per_age	0.8780	+0.3722
traffic_x_hour	0.5076	+0.0019
vehicle_x_multi	0.5036	-0.0022
weather_x_traffic	0.5023	-0.0035
rating_x_traffic	0.5008	-0.0050
(baseline)	0.5058	0.0000

Section F. Interpretation — what does it MEAN?

Q. In one paragraph: what did you find?

A. Zomato's **operational behaviour** — riders, customers, traffic patterns, weather impact, festival behaviour, vehicle mix, order types, multi-delivery patterns — is statistically indistinguishable North vs South in our data. The ONE non-geographic feature that does carry regional signal is the haversine distance between pickup and drop, whose distribution shape differs significantly (KS $p \approx 10^{-10}$) even though median and IQR match. This isn't behaviour — it's **urban form**: South Indian cities have a structurally different layout of restaurants relative to customers than North Indian cities. The model surfaces this without needing absolute coordinates.

Q. Walk me through the three-layer finding.

A. **Layer 1 — Behaviour** (AUC ≈ 0.51 , random): 11 univariate tests on individual delivery features all $p > 0.05$; six stress tests confirm no signal. Zomato runs uniformly across India. **Layer 2 — Structure** (StratKFold AUC ≈ 1.00 , GroupKFold AUC ≈ 0.91): adding haversine pickup-to-drop distance lifts AUC enormously. The distance distribution itself is regionally distinctive — that's geometric, not behavioural. **Layer 3 — Geography** (AUC = 1.00, perfect by construction): raw lat/lon coordinates trivially separate regions because we defined regions by state, which is a function of coordinates.

Q. Why is distance considered 'structural' and not 'geographic'?

A. Geographic features = absolute location (lat, lon). Structural features = scalars derived from the layout but not the location itself. distance_km is the haversine between TWO points within a single order; two orders from different cities can have identical distance_km. If we were memorising city locations, distance_km wouldn't generalise across held-out cities — but GroupKFold AUC stays at 0.911, proving the signal is in the *distribution* of distances, not the absolute coordinate values.

Q. Why did Fold 4 of GroupKFold drop to 0.642?

A. Fold 4 held both Kochi (Kerala) and Hyderabad (Telangana) at the same time. The remaining South training cities were all Karnataka (BANG, MYS) and Tamil Nadu (CHEN, COIMB). The model learned 'South = Karnataka/TN distance profile' and didn't fully generalise to Kerala+Telangana. **This is a real limitation worth flagging**: South India is internally heterogeneous, confirmed by the per-state classifier (38% vs 25% random) in Section E test 5.

Q. Why is the temporal leak (month) a real concern and not just noise?

A. Calendar date range is identical for both regions (Feb 11 – Apr 6 2022). But within-region *density* differs sharply:

Month	North %	South %
February	20.8%	4.3%
March	66.0%	79.9%
April	13.2%	15.8%

Feb is 4.8× more represented in North orders. If you include month as a feature, the model just learns 'February $\Rightarrow$ North' and inflates AUC from 0.51 to 0.59. We dropped month and day_of_week to remove this proxy.

Section G. Adaptability (judging criterion 2)

Q. How adaptable is your system to new data?

A. Five concrete scenarios, all already supported in the codebase:

Scenario	Change required	File / function
A new city joins the dataset (e.g. JAWHAR) → add to CITY_CODE_MAP and CITY_CODES and Region.py:13-78	Add new city to CITY_CODE_MAP and CITY_CODES	src/region.py:13-78
An unknown city prefix arrives in production → function change. _classify_by_coords() falls back to default _classify_by_coords	Function change. _classify_by_coords() falls back to default _classify_by_coords	src/region.py:13-78
Brief redefines South (e.g. add AP, Andhra Pradesh) → SOUTH_STATES set, 1 line. Re-run pipeline	Re-define SOUTH_STATES set, 1 line. Re-run pipeline	src/region.py:80
Want to add a feature (e.g. live traffic) → Add a column in src/features.py::add_features; pipeline picks it up automatically.	Add a column in src/features.py::add_features; pipeline picks it up automatically.	src/features.py
Re-run on a completely new CSV dump → python notebooks/_run_regional.py — finishes notebooks/_refresh_region.py, deck and PDFs automatically	python notebooks/_run_regional.py — finishes notebooks/_refresh_region.py, deck and PDFs automatically	notebooks/_run_regional.py
Deploy the trained model for live inference → Deploy(model_file=...) loads in <100 ms, models/region_classifier_v2_structural.txt	Deploy(model_file=...) loads in <100 ms, models/region_classifier_v2_structural.txt	models/region_classifier_v2_structural.txt
Swap the ML framework (e.g. to XGBoost) → X, y, same cat features; ~5-line edit. Cat features are checked already in deploy	Swap the ML framework (e.g. to XGBoost) → X, y, same cat features; ~5-line edit. Cat features are checked already in deploy	notebooks/_refresh_region.py

Q. How long does the full pipeline take on fresh data?

A. End-to-end on a CPU laptop: ~3 minutes for region label + 6 stress tests + 4 trained models + SHAP plots. Deck and PDF rebuild from JSON in ~10 seconds. Cold-start to publishable result in < 5 minutes total.

Q. Is the model overfit to the 22 cities we have?

A. Two pieces of evidence say no. (1) **GroupKFold by city**: behaviour-only stays at 0.506 random (so any would-be over-fit per-city pattern doesn't generalise → there isn't one). (2) **Structural model GroupKFold**: AUC stays at 0.911 on held-out cities, meaning the distance-distribution signal transfers. The Fold-4 dip to 0.642 IS a real generalisation limit — we report it honestly.

Q. Could the same approach work for a different country?

A. Yes. Replace CITY_CODE_MAP with the equivalent ID prefix dictionary for the new country, update SOUTH_STATES (or rename to whatever region split applies), redefine the bounding-box fallback in _classify_by_coords. Everything else — cleaning, features, modelling, stress tests, deck builder — is country-agnostic.

Section H. Technical defense — expected attacks

Q. Could you have over-fit by tuning hyperparameters on test data?

A. No. Hyperparameters were chosen once before training and held fixed across all six stress tests. There is no separate test set we tuned to — we run K-fold CV and report the mean. Same hyperparameters across Model B, Model B', Model C, and the GroupKFold variant.

Q. How do you know the GroupKFold isn't just unlucky?

A. Five folds, AUC values 0.998, 0.998, 1.00, 0.642, 0.917 for the structural model. The dip is concentrated in one fold (the one holding both Kerala and Telangana). On the other four folds the model generalises near-perfectly. We report the mean (0.911) AND call out the fold-4 anomaly explicitly.

Q. Why not use deep learning?

A. Three reasons. (1) Tabular data with 45k rows is well in LightGBM's sweet spot; deep nets don't beat it here and are harder to tune in the time we had. (2) LightGBM trains in seconds, allowing us to run six stress tests instead of one. (3) Interpretability — SHAP on LightGBM is exact and fast; on deep nets it's approximate.

Q. Did you check for label imbalance distortion?

A. Yes, three ways. (1) `class_weight='balanced'` in the LGBM classifier. (2) Reported balanced accuracy (0.964) and F1 (0.952) alongside AUC. (3) StratifiedKFold keeps the 39/61 ratio within every fold.

Q. How do you know SHAP is trustworthy on LightGBM?

A. TreeExplainer for tree ensembles is **exact** (not approximate) and runs in polynomial time. Shapley values give the unique additive feature attribution satisfying efficiency, symmetry, dummy, and additivity axioms (Lundberg & Lee, 2017). We use mean absolute SHAP per feature as the importance metric — less prone to bias than LightGBM's gain importance.

Section I. Limitations and honest caveats

Q. What are the limits of this study?

A. Six honest caveats:

- **Window:** Only 8 weeks of data (Feb 11 – Apr 6 2022). Doesn't cover Onam, Pongal, Diwali — festivals that COULD reveal regional differences.
- **City coverage:** 22 cities, with South sample weighted toward Karnataka + Tamil Nadu. Kerala has only 701 orders; Andhra Pradesh has none.
- **Sub-regional balance:** Within South we cannot confidently distinguish Kerala+Telangana behaviour from Karnataka+TN behaviour (Fold-4 anomaly).
- **No live traffic / weather intensity:** Categorical weather + traffic miss numeric severity that might carry signal.
- **No temporal split:** Generalisation to a future week is unverified.
- **Probabilities not calibrated:** `class_weight='balanced'` means raw posterior probabilities are NOT well-calibrated — they're ordered correctly (high AUC) but the 0.5 threshold isn't operationally optimal.

Q. If you had 6 more hours, what would you do?

A. (1) Add restaurant-prep-time rolling history if a `restaurant_id` arrives. (2) Run a temporally held-out test (last week as test, prior weeks as train). (3) Calibrate probabilities with isotonic regression on OOF preds.

(4) Fit a quantile regressor for delivery-time intervals (rather than point predictions) on the ETA task. (5) Try a city-conditioned random effects model — does region beyond city tell us anything? (6) Bring in zonal demographic data (population density, road network length) — likely the structural signal source.

Q. What would convince you that there IS a regional behavioural difference?

A. Either (a) behaviour-only GroupKFold AUC > 0.6 with at least one non-temporal feature carrying SHAP > 0.5 , or (b) a univariate test with effect size (Cramér's V or KS D) > 0.15 on at least one rider/customer/weather/traffic feature. We saw neither — only distance_km clears that effect-size bar ($D=0.088$ is borderline; and distance is structural, not behavioural).

If you remember three lines from this entire document, make them these:

1. **Geo AUC = 1.000** — labels are clean.
2. **Behaviour-only AUC = 0.508** — behaviourally Zomato is uniform across India.
3. **Structural AUC (GroupKFold) = 0.911** — urban GEOMETRY differs, behaviour doesn't.

Repo: github.com/TheClazer/Zomato-delivery-estimation · **Built from:** `live reports/regional_findings.json` · **Generator:** `notebooks/_build_qna_elite.py`